

Testing as Specification: A Pedagogical Paradigm for Requirement Engineering in AI-Assisted Programming

Hudson Mitchell-Pullman

The University of California, San Diego

Abstract. Recent work in Artificial Intelligence in Education (AIED) focuses on prompt-engineering pedagogy, which often lacks academic merit. Beyond prompt engineering, researchers have not widely investigated how AI-assisted programming is changing student learning objectives. Including well-specified requirements in prompts to AI-coding assistants improves code quality. Crucially, prior work demonstrates that software testing forces students to identify and specify requirements: a skill we argue translates to prompting skills for AI-assisted programming. However, AIED literature has not recognized the pedagogical connection between requirement engineering and software testing. The comprehensive capabilities of AI coding assistants enable previously non-technical students to pursue computer science through "vibe coding," necessitating new teaching methodologies for requirement engineering. Previous work fails at incorporating Large Language Models (LLMs) into a testing pedagogy that scaffolds requirement engineering. Thus, we propose using software testing as a pedagogical vehicle to teach requirements engineering. We propose using Large Language Models (LLMs) to simulate help-seeking behaviors, enabling students to learn software testing by teaching the LLM. The LLM iteratively probes requirements by asking the real student to develop and define test-case inputs and outputs from ill-defined system descriptions. The primary contributions of this paper include: a theoretically grounded paradigm that improves requirement engineering pedagogy by teaching software testing; and practical suggestions for creating and testing AI-supported pedagogical systems for requirement engineering grounded in software testing theory.

Keywords: AI-assisted software development · End-user programming · Computer science education · Requirement engineering · Software testing

1 Introduction

Large Language Models (LLMs) are now able to solve nearly all introductory programming assignments and code complicated software systems [4,27]. The primary way that users interface with AI-coding assistants is through prompts [34], which are written in natural language. Specification errors in LLM prompts degrade code output quality [36], thus motivating work on teaching prompt

engineering in computer science courses. Since LLMs can write code when provided complete requirements [10] but struggle when prompts are missing or conflating specifications [36], computer science education should include more targeted practice to improve requirement elicitation, the process of researching and discovering software requirements, and articulation. Recent literature in the AIED community focuses on teaching prompt engineering [23,20,35], with much research focusing on "ad hoc prompt engineering" [36] for unambiguous problems. Regrettably, much of this work fails to consider past work grounding requirement engineering education in cognitive science. Before LLMs, it remained challenging to teach requirement engineering or "end-user programming" [6] because software development still required a low-level understanding of a programming language and its syntax. However, we hypothesize LLMs enable us to now use software testing to scaffold requirement engineering education by emphasizing edge case identification, specification probing, system testing, and natural language articulation. This presents an opportunity for a new pedagogical requirement engineering paradigm that teaches the cognitive skills for prompt engineering based on existing requirement engineering frameworks [18,25].

This work introduces a novel paradigm in requirement engineering pedagogy that uses software testing to facilitate requirement elicitation. AI-assisted programming demands specific requirement engineering to improve prompt specificity [37] skills that are present in software testing pedagogy [14,8]. Instead of explicitly teaching students to follow a rigid requirement engineering framework [18], the testing-as-specification paradigm uses the testing as a pedagogical vehicle for requirement engineering. Based on exploratory testing [13,14], students will define data classes, interactions across the data classes, and update the system requirements based on the outputs of these tests. In this paradigm, we propose using a ride share matching simulation for the testing environment (Figure 1). This scenario is necessarily suitable for the testing-as-specification paradigm because it can be displayed visually [16], involves multiple user types [17], is familiar to many students, and involves many implicit requirements¹. Instead of forcing students to write software tests in a programming language, we aim to make the paradigm accessible to end-user programmers [6] who may be unfamiliar with a particular programming language. Requirement engineering is a highly iterative process that informs iteration based on software tests. Additionally, the testing paradigm uses the teachable agent framework [15,24] whereby the LLM simulates a help-seeking student and the real student acts as a TA by helping the LLM student to develop, interpret, and iterate upon software tests and integrate the findings into the requirement specifications. We argue the testing-as-specification paradigm will both improve the efficacy of requirement engineering education and increase engagement in CS1 courses that have traditionally focused on syntax memorization [21]. This work improves upon past requirement engineering pedagogy by using software testing to probe and iterate system requirements.

¹ <https://www.uber.com/us/en/marketplace/matching/>

2 A Review of Requirement Engineering and Testing

In this section, we synthesize existing literature on requirement engineering and its implications in AI-assisted programming. First, we investigate seminal requirement engineering frameworks in business and pedagogical contexts. Next, we examine how requirement engineering has evolved, and the role it has in improving engagement and equity in introductory computer science courses among end-user programmers. Finally, we examine AI-assisted programming and the roles that requirement engineering plays in AI-assisted code development via prompt engineering.

2.1 Requirement Engineering Pedagogy

As LLMs can now solve many CS1 problems and build complex software systems[4], prompt engineering is becoming an increasingly important skill. Students now interface with AI-coding assistants[24] using natural language instructions that give the LLM constraints for what to code. To address prompt engineering pedagogy, past works propose teaching tactical prompt engineering frameworks [20,35] and prompting revision techniques[3]. However, Ma et. al draws parallels between requirement engineering and prompt engineering, showing that requirement-based training improves LLM output quality by 12.7 percent when compared with the "business-as-usual" prompt engineering pedagogy[23]. Despite the importance of requirement engineering, especially as AI-assisted programming becomes ubiquitous, requirement engineering is seldom taught in CS1 courses[7]. Even when requirement engineering is taught, professors approach it within an industry context[7], where students consider the viewpoints of stakeholders[30] and design requirements according to the concerns of the stakeholders. Additionally, requirement engineering pedagogy is largely decentralized and unsystematic[7]. It has been challenging to develop an accurate model of requirement engineering, as the process is highly iterative[9] and combines many different cognitive processes.

However, AI-assisted programming is enabling non-technical end-user programmers to build systems that they use internally, meaning that requirement engineering does not rely on external stakeholders and their goals. As such, requirement engineering pedagogy should reflect this shift in learning objectives by teaching students to define *functional requirements*, a set of requirements describing how a system should behave under a set of constraints. According to the IEEE, "high quality requirements should be unambiguous, complete, and consistent" [1]. LLM output quality drops precipitously when requirements are left unspecified, with an average of a 22.6 percent drop in accuracy[36] due to requirement omission errors in prompts. An additional requirement engineering framework is Goal-Oriented Requirement Engineering (GORE)[18], a paradigm that decomposes high-level system goals into functional requirements that define the behaviors of a system. Specifically, each goal is broken down into its components, which are then analyzed and converted into testable requirements that are then evaluated using software tests. To further probe requirements, GORE uses obstacles to test edge cases and avoid the "happy path" bias that often occurs

during requirement engineering[19]. While some of the GORE methods mirror software testing, section 2.2 discusses how software engineers typically test their code and how it differs from GORE.

2.2 Software Testing in CS Education

Computer science education researchers have emphasized the importance of test-driven learning (TDL)[14] for decades. In TDL, students write their code's test cases before implementing the code[8]. Software testing is often thought of as separate from requirement engineering[7], and to the best of our knowledge, has not been directly used to support requirement engineering for AI-assisted programming. This reveals a large gap in existing AIED literature, as previous research demonstrates that "problem and program comprehension strategies and abilities appear to be intricately tied to students' abilities to devise useful test cases"[29]. Thus, when we analyze how to improve requirement engineering pedagogy, it naturally makes sense to consider the role of software testing. In a recent study analyzing how students engineer test cases based on test output feedback[29], students often wrote new software tests without considering the results of their previous tests. Additionally, nearly none of the participants used the results of the tests to confirm their understanding of the specifications. These results align with empirical observational studies of professional software developers, showing that they often write tests based on their code syntax rather than the GORE framework or a systematic analysis of the system specifications [2].

Finally, Problem-Oriented Programming and Testing (POPT)[22] uses ill-defined specifications to challenge students' software testing abilities. POPT requires students to write input-output pairs test cases before and during code development, which "help the student to enlighten the problems' specifications as well as improve the quality of generated code"[22]. Exploratory testing modifies this paradigm by encouraging students to iteratively create software tests based on the results of previous tests[13]. That is, instead of writing individual tests in isolation, the student should continuously probe implicit requirements through their tests, which are used to further specify the code specifications and the design of future tests. Additionally, exploratory testing emphasizes system-level testing, where the tester analyzes the interactions between multiple users and attributes in addition to branch testing the attributes individually[12]. If exploratory testing is combined with TDL, it can lead to the identification of additional requirements during requirement engineering.

3 Implications for AI-Assisted Programming Education

In the following sections, we outline the implications of requirement engineering and software testing in computer science education for AI-assisted programming. Specifically, we begin by detailing an example of software testing within a multi-user system and its parallels with requirement engineering. Then, we use the example to generalize our testing-as-specification paradigm to multiple problem

types that are grounded in computer science pedagogy. We also detail how researchers can use AI to augment software testing to probe requirements.

3.1 Requirement Engineering for Ride Share Matching

We present a scenario demonstrating software testing and edge case exploration and its implications on requirement engineering pedagogy using a ride share matching simulation (to simplify this example, we assume that all ride share vehicles are autonomous). To develop a mental schema of the matching logic, developers must write test cases that encapsulate the data attributes and possible edge cases of the system. If a novice computer science student were to instruct an LLM to construct the matching logic for this ride share platform, we expect the student to focus on the *structure* of the matching system while omitting the *system’s behavior and function* [11]. Previous work shows novice students can correctly identify high-level structural components of a system, such as the autonomous vehicles and riders. If students followed a requirement engineering framework, which are often omitted from introductory computing courses [7], they would likely begin with defining a system goal (i.e., reducing wait time and the distance a vehicle travels to pick up a rider) and systematically reducing the level of abstraction to define more concrete requirements [18]. However, experts devote more cognitive effort to problem formulation and high-level planning than novices in a CS1 context [33,5,31,32], meaning they would likely struggle in developing a schema that accurately represented the system requirements. Specifically, novices often understand the structures of a complex system, such as the vehicle and the rider in a ride share platform, but fail to identify their behaviors and functions when discussing the problem space [11].

These findings reveal that if a novice were attempting to define requirements for a ride share matching system, they would likely struggle to consider the interactions and the causal relationships between vehicles and riders. Suppose a novice instructs an AI-coding assistant to write the following requirement:

“compare the locations of the rider and the vehicle and match the nearest vehicle with the nearest rider.”

This seems like a plausible requirement, however, it fails to consider all of the different attributes that have causal effects on the matching outcome in a ride share platform. From this requirement, we can only confirm that the student has considered the current locations of the rider and the vehicle as requirements and test inputs in their ride share matching system, omitting many user attributes necessary in accurately matching riders to vehicles (Table 1).

To further demonstrate the weaknesses of this requirement, consider the following software test written in natural language:

There is only one vehicle currently active in the system. It is currently occupied with passengers. Rider 1 is located five blocks from the current location of the vehicle while it travels to its destination. Rider 2 is located three blocks away from the current location of the vehicle. Rider 2 requested

Table 1. Rideshare system object fields. ✓ = captured by the naive “compare locations” requirement; × = omitted fields surfaced through testing.

Rider Field		Demonstrated Vehicle Field	Demonstrated
origin (pickup)	✓	current_location (GPS)	✓
destination (dropoff)	✓	destination (route)	×
accessible (boolean)	×	is_occupied (boolean)	×
request_time (timestamp)	×	is_assigned (boolean)	×
eta_dropoff (estimated)	×	is_accessible (capability)	×

Table 2. Functional requirements implicitly encoded in the natural-language test case. Each row is surfaced by analyzing a single input-output pair.

ID Requirement
R1 Vehicle shall accept a new rider assignment while occupied and en route.
R2 Assignment priority shall respect rider request timestamp (FIFO) and match distance.
R3 Vehicle routing shall account for the current drop-off before proceeding to a new pickup.
R4 Vehicle shall support sequential multi-passenger trips in a single dispatch cycle.
R5 System shall minimise total vehicle travel distance across all pickups and drop-offs.
R6 Vehicle shall update its assignment queue en route without re-dispatch.

a ride at 13:10, while Rider 1 requested a ride ten minutes earlier at 13:00. The vehicle is currently occupied and en route to a destination that is one block north of Rider 1’s pickup location. The destination of Rider 1 is two blocks away from Rider 2’s pickup location.

In a ride share matching system that contains the goals of effectively minimizing wait time, vehicle travel distance, and down time, we would evidently expect the vehicle to:

Assign itself to Rider 1 while en route to its drop off location, drop off its current passengers, pick up Rider 1, assign itself to Rider 2, drop off Rider 1, pick up Rider 2.

This singular input-output pair contains multiple requirements, which can be visualized in Table 2

If we repeat this process, that is, considering different attributes, defining a scenario in which multiple attributes interact, and explicitly stating the expected behavior of the matching system, we will eventually compile a list of requirements and user attributes that more comprehensively define the problem domain.

This scenario is particularly complex because it involves multiple attributes: whether the vehicle is occupied, the pickup and drop off locations of the riders, the distance between the riders, and the times that the rider requested the vehicle. If a novice were to develop this test case, it would first require them to develop a schema of the ride share matching system that includes all of these attributes, thus surpassing the comprehensiveness of the initial requirement. Additionally, when writing the test case, the student must consider the causal effects that the wait time and vehicle drop off location have on the matching algorithm, requiring them to consider the behaviors of these attributes. When students omit the test case and directly attempt to define requirements, they forego the scaffolding provided by the software testing.

This presents an apparent problem, as previous literature demonstrates that novices struggle with reasoning about the behavioral and functional aspects of complex systems. Thus, it is unreasonable to assume that students will be able to perform complex requirement engineering by beginning with an overarching goal (e.g., minimize wait time) and progressively increasing the granularity of the goal by considering user attributes and their behaviors and functions. Thus, we propose using software testing as a pedagogical vehicle to scaffold requirement engineering. In section 3.2, we highlight previous works and provide a theoretical basis for the testing-as-specification paradigm to support requirement engineering through software testing.

3.2 Using AIED to Facilitate Software Testing and Requirement Engineering

The ride share system in section 3.1 illustrates the general pattern in using software tests to probe requirements; when students develop test cases for an ill-defined system, such as the ride share system with the vague initial requirement, they implicitly produce requirements through software testing. The testing-as-specification interface used to scaffold this process is shown in Figure 1. In Section 2, we highlighted the gaps in existing requirement engineering pedagogy. Previous research has further identified this gap, and [29] concluded that "problem and program comprehension strategies and abilities appear to be intricately tied to students' abilities to devise useful test cases."

Recent research has combined aspects of POPT and requirement engineering, but to our knowledge, no previous literature has explicitly proposed a paradigm using software testing with an LLM to facilitate requirement engineering. We have identified three recent papers as foundational to our proposed paradigm. Ma et al. used an LLM to probe hidden requirements for interactive games and prompt program[?] by instructing the LLM to guide the student to identifying implicit requirements[23]. Although this work makes important claims connecting requirement engineering with prompt engineering, it fails to effectively use an LLM to scaffold software testing based on a well-established testing framework. Similarly, Ma et al. used the teachable agent[15] framework to scaffold unit testing. In the teachable agent framework, an LLM simulates a student, requiring the real human student to act as a TA and "teach" the LLM, which supports previous

work emphasizing the efficacy of learning-by-teaching[28]. This work showed promising results in improving hypothesis testing, but did not use the testing to inform requirement engineering. Additionally, the tests were primarily unit tests that did not involve multiple interacting users or attributes, making it impossible to use the exploratory testing paradigm. Finally, "Probeable Problems"[26] propose a new competitive computer science paradigm that intentionally omits specifications, definitions, and edge cases, similar to the POPT framework. Instead of seeing the pre-defined requirements, students must use the "Oracle" to test different inputs and use the Oracle outputs to inform the design of their system. Specifically, "Probeable Problems" flip the traditional paradigm where students must implement pre-defined edge cases in code by omitting the edge cases and requiring the students to discover them while allowing students to use an LLM to actually implement the code. While this paradigm is similar to POPT and showed promising requirement engineering outcomes, it did not include a pedagogical scaffold for software testing due to its competitive nature. Thus, our paradigm must adapt this testing method and include necessary scaffolding to support software testing.

Our testing-as-specification paradigm synthesizes and extends these three contributions by using exploratory software testing as the primary mechanism through which students discover and articulate requirements. Where ROPE uses declarative requirement documents and HypoCompass uses test cases to diagnose existing bugs, our paradigm treats the act of writing a test case as itself a requirement specification activity: each natural language input-output pair implicitly encodes functional requirements that the student must then extract and formalize. This mirrors the GORE framework [18], in which high-level goals are decomposed into testable requirements through obstacle analysis, but grounds the decomposition in concrete test scenarios rather than abstract goal hierarchies. From POPT [22] and Probeable Problems [26], we inherit the use of ill-defined specifications that force students to probe implicit requirements, but we add structured pedagogical scaffolding through the teachable agent framework [15], in which the LLM simulates a help-seeking student whose questions iteratively surface missing attributes, edge cases, and interaction effects that the real student must resolve through additional tests. Critically, our paradigm requires multi-user, system-level test scenarios (Section 3.1) that involve interactions across data classes [17], moving beyond the single-function unit tests used in HypoCompass toward the exploratory testing paradigm [13] in which each test outcome informs the design of the next test and progressively refines the requirement specification. By combining exploratory testing with the teachable agent framework within an ill-defined problem space, the testing-as-specification paradigm provides the scaffolding absent from Probeable Problems, the requirement engineering connection absent from HypoCompass, and the testing mechanism absent from ROPE.

Testing Paradigm for Requirement Engineering in AI-Assisted Programming



Fig. 1. The testing-as-specification interface. *Left:* dialogue log between the LLM-simulated student (TA role) and the real learner, showing requirement fields being surfaced iteratively. *Center:* visual ride share grid showing vehicle and rider positions with live test output. *Right:* iterative edge case testing panel and natural-language test specification, showing how students discover and define system requirements through testing.

4 Conclusion

To better equip end-user programmers with the necessary skills to engage with AI-coding assistants, we must use software testing as a pedagogical scaffold for requirement engineering. By using LLMs to facilitate multi-user software testing, we may not just improve requirement engineering, but we may also help to increase equity and diversity in the computing field. Many students taking computer science courses, particularly those who are not majoring in computer science, care more about developing a computer science vocabulary, know as “conversational programming,”[6] and LLMs now enable computer science courses to better serve these populations while teaching them core computational thinking skills that will improve their end-user programming abilities with AI.

References

1. IEEE recommended practice for software requirements specifications. Tech. Rep. Std 830-1998, IEEE (1998). <https://doi.org/10.1109/IEEESTD.1998.88286>
2. Aniche, M., Treude, C., Zaidman, A.: How developers engineer test cases: An observational study. *IEEE Transactions on Software Engineering* **48**(12), 4925–4946 (2022). <https://doi.org/10.1109/TSE.2021.3131529>
3. Arawjo, I., Swoopes, C., Vaithilingam, P., Wattenberg, M., Glassman, E.L.: Chainforge: A visual toolkit for prompt engineering and llm hypothesis testing. In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM (2024). <https://doi.org/10.1145/3613904.3642016>
4. Becker, B.A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J., Santos, E.A.: Programming is hard – or at least it used to be: Educational opportunities and challenges of ai code generation. In: *Proceedings of the 54th ACM Technical Symposium on Computer Science Education*. pp. 500–506. ACM (2023). <https://doi.org/10.1145/3545945.3569759>
5. Chi, M.T.H., Feltovich, P.J., Glaser, R.: Categorization and representation of physics problems by experts and novices. *Cognitive Science* **5**(2), 121–152 (1981). https://doi.org/10.1207/s15516709cog0502_2
6. Cunningham, K., Blanchard, J., Ericson, B., Guzdial, M.: Avoiding the turing tarpit: Learning conversational programming by starting from code’s purpose. In: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM (2021). <https://doi.org/10.1145/3411764.3445571>
7. Daun, M., Grubb, A.M., Stenkova, V., Tenbergen, B.: A systematic literature review of requirements engineering education. *Requirements Engineering* **28**, 145–175 (2023). <https://doi.org/10.1007/s00766-022-00381-9>
8. Edwards, S.H.: Improving student performance by evaluating how well students test their own programs. In: *Journal on Educational Resources in Computing*. vol. 3 (2003). <https://doi.org/10.1145/1029036.1007534>
9. Enoiu, E.P., Tukseferi, G., Feldt, R.: Towards a model of testers’ cognitive processes: Software testing as a problem solving approach. In: *IEEE International Conference on Software Quality, Reliability and Security Companion*. pp. 272–279 (2020). <https://doi.org/10.1109/QRS-C51114.2020.00053>
10. Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Lahiri, S.K.: Llm-based test-driven interactive code generation: User study and empirical evaluation.

- IEEE Transactions on Software Engineering (2024). <https://doi.org/10.1109/TSE.2024.3428972>
11. Hmelo-Silver, C.E., Pfeffer, M.G.: Comparing expert and novice understanding of a complex system from the perspective of structures, behaviors, and functions. *Cognitive Science* **28**, 127–138 (2004). https://doi.org/10.1207/s15516709cog2801_7
 12. Itkonen, J., Mäntylä, M.V., Lassenius, C.: The role of the tester’s knowledge in exploratory software testing. *IEEE Transactions on Software Engineering* **39**(5), 707–724 (2013). <https://doi.org/10.1109/TSE.2012.55>
 13. Itkonen, J., Rautiainen, K.: Exploratory testing: A multiple case study. In: *International Symposium on Empirical Software Engineering*. IEEE (2005). <https://doi.org/10.1109/ISESE.2005.1541817>
 14. Janzen, D.S., Saiedian, H.: Test-driven learning in early programming courses. In: *Proceedings of the 39th SIGCSE Technical Symposium on Computer Science Education*. pp. 532–536 (2008). <https://doi.org/10.1145/1352135.1352315>
 15. Jin, H., Lee, S., Shin, H., Kim, J.: Teach ai how to code: Using large language models as teachable agents for programming education. In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM (2024). <https://doi.org/10.1145/3613904.3642349>
 16. Ko, A.J., Abraham, R., Beckwith, L., Blackwell, A., Burnett, M., Erwig, M., Scaffidi, C., Lawrance, J., Lieberman, H., Myers, B., Rosson, M.B., Rothermel, G., Shaw, M., Wiedenbeck, S.: The state of the art in end-user software engineering. *ACM Computing Surveys* **43**(3), 1–44 (2011). <https://doi.org/10.1145/1922649.1922658>
 17. Kuhn, D.R., Wallace, D.R., Gallo Jr., A.M.: Software fault interactions and implications for software testing. *IEEE Transactions on Software Engineering* **30**(6), 418–421 (2004). <https://doi.org/10.1109/TSE.2004.24>
 18. van Lamsweerde, A.: Goal-oriented requirements engineering: A guided tour. In: *Proceedings of the 5th IEEE International Symposium on Requirements Engineering*. pp. 249–262. IEEE (2001). <https://doi.org/10.1109/ISRE.2001.948567>
 19. van Lamsweerde, A., Letier, E.: Handling obstacles in goal-oriented requirements engineering. In: *IEEE Transactions on Software Engineering*. vol. 26, pp. 978–1005 (2000). <https://doi.org/10.1109/32.879820>
 20. Lo, L.S.: The CLEAR path: A framework for enhancing information literacy through prompt engineering. *The Journal of Academic Librarianship* **49**(4) (2023). <https://doi.org/10.1016/j.acalib.2023.102720>
 21. Loksa, D., Ko, A.J., Jernigan, W., Oleson, A., Mendez, C.J., Burnett, M.M.: Programming, problem solving, and self-awareness: Effects of explicit guidance. In: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. pp. 1449–1461. ACM (2016). <https://doi.org/10.1145/2858036.2858252>
 22. Lustosa Neto, V., Coelho, R., Leite, L.S., Guerrero, D.S., Mendonça, A.P.: Popt: A problem-oriented programming and testing approach for novice students. In: *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. pp. 1099–1108. IEEE (2013). <https://doi.org/10.1109/ICSE.2013.6606660>
 23. Ma, Q., Peng, W., Yang, C., Shen, H., Koedinger, K.R., Wu, T.: What should we engineer in prompts? training humans in requirement-driven llm use. *ACM Transactions on Computer-Human Interaction* (2025). <https://doi.org/10.1145/3731756>
 24. Ma, Q., Wu, T., Koedinger, K.R.: Is ai the better programming partner? human-human pair programming vs. human-ai pair programming. *arXiv preprint arXiv:2306.05153* (2023). <https://doi.org/10.48550/arXiv.2306.05153>

25. Nuseibeh, B., Easterbrook, S.: Requirements engineering: A roadmap. In: Proceedings of the Conference on The Future of Software Engineering. pp. 35–46. ACM (2000). <https://doi.org/10.1145/336512.336523>
26. Pawagi, M., Kumar, V.: Probeable problems for beginner-level programming-with-ai contests. In: Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education (2024). <https://doi.org/10.1145/3632620.3671108>
27. Prather, J., Denny, P., Leinonen, J., Becker, B.A., Albluwi, I., Craig, M., Keuning, H., Kiesler, N., Kohn, T., Luxton-Reilly, A., MacNeil, S., Petersen, A., Pettit, R., Reeves, B.N., Šavelka, J.: The robots are here: Navigating the generative AI revolution in computing education. In: Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education. pp. 108–159. ACM (2023). <https://doi.org/10.1145/3623762.3633499>
28. Shen, H., Koedinger, K.R., Wu, T.: Hypocompass: Large-language-model-based tutor for hypothesis construction in debugging. In: Proceedings of the 2024 ACM Conference on Learning @ Scale (2024)
29. Shin, A.M., Kazerouni, A.M.: A model of how students engineer test cases with feedback. *ACM Transactions on Computing Education* **24**(1), 1–31 (2024). <https://doi.org/10.1145/3628604>
30. Sommerville, I., Sawyer, P.: Viewpoints: Principles, problems and a practical approach to requirements engineering. *Annals of Software Engineering* **3**, 101–130 (1997)
31. Song, T., Becker, K.: Expert vs. novice: Problem decomposition/recomposition in engineering design. In: 2014 International Conference on Interactive Collaborative Learning (ICL). pp. 181–190. IEEE (2014)
32. Song, T., Becker, K., Gero, J., DeBerard, S., DeBerard, O., Reeve, E.: Problem decomposition and recomposition in engineering design: A comparison of design behavior between professional engineers, engineering seniors, and engineering freshmen. *Journal of Technology Education* **27**(2), 37–56 (2016)
33. Srinath, S., Vadaparty, A., Smith IV, D.H., Porter, L., Zingaro, D.: Assessing problem decomposition in cs1 for the genai era. In: Proceedings of the 56th ACM Technical Symposium on Computer Science Education (2025)
34. Vadaparty, A., Zingaro, D., Smith IV, D.H., Padala, M., Alvarado, C., Gorson Benario, J., Porter, L.: Cs1-llm: Integrating llms into cs1 instruction. In: Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education (2024). <https://doi.org/10.1145/3649217.3653584>
35. Wang, T., Zhou, N., Chen, Z.: Enhancing computer programming education with LLMs: A study on effective prompt engineering for Python code generation. *arXiv preprint arXiv:2407.05437* (2024). <https://doi.org/10.48550/arXiv.2407.05437>
36. Yang, C., Shi, Y., Ma, Q., Liu, M.X., Kästner, C., Wu, T.: What prompts don’t say: Understanding and managing underspecification in llm prompts. In: Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems. ACM (2024)
37. Zi, Y., Menon, H., Guha, A.: More than a score: Probing the impact of prompt specificity on llm code generation. In: Proceedings of the Association for Computational Linguistics (2025). <https://doi.org/10.48448/599z-yh65>